UNIVERSIDADE ESTADUAL DO SUDOESTE DA BAHIA – UESB

DEPARTAMENTO DE CIÊNCIAS EXATAS – DCET | CURSO DE CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: COMPILADORES (2026.2) – PROF. JOSÉ CARLOS MARTINS OLIVEIRA

DISCENTE: GUILHERME DIAS SOUSA

RELATÓRIO TÉCNICO E DOCUMENTAÇÃO DO TRABALHO PRÁTICO DA I UNIDADE
ANALISADOR LÉXICO DA LINGUAGEM MINI_PASCAL

1. Introdução e Visão Geral

Este trabalho apresenta o desenvolvimento do **Analisador Léxico para a linguagem Mini_Pascal**, primeira etapa do projeto de construção de compiladores da UESB. O sistema foi implementado na linguagem **C# (.NET 10.0)**, utilizando a plataforma **Windows Forms** para prover uma interface visual rica, interativa e amigável. O núcleo do analisador baseia-se estritamente no **Autômato Finito Determinístico (AFD)** formalmente projetado, garantindo tempo de resposta linear *O(n)* em relação ao tamanho do código-fonte e recuperação robusta de erros léxicos.

2. Instruções de Instalação e Execução

A) Execução Direta (Sem necessidade de IDE ou recompilação):
O aplicativo foi compilado em modo Release e disponibilizado na pasta executavel/ na raiz do projeto. Para executá-lo no Windows, basta dar um duplo clique no arquivo **MiniPascalLexer.exe**.
B) Recursos da Interface Gráfica:
• **Editor de Código:** Permite digitação direta ou colagem de código-fonte.
• **Menu Exemplos de Teste:** Carrega com um clique qualquer um dos 5 casos de teste pré-definidos.
• **Botão 'Executar Análise Léxica':** Processa o texto via AFD e popula instantaneamente a tabela de tokens.
• **Tabela de Tokens:** Exibe colunas ordenadas com Índice, Linha, Coluna, Lexema e Classificação do Token.
• **Painel de Diagnósticos:** Informa o total de tokens válidos e lista detalhada de erros léxicos encontrados.
• **Botão 'Salvar Tokens (.txt)':** Exporta o arquivo de saída no formato exato solicitado: <lexema, token>.
C) Execução via Linha de Comando (Modo Batch):
MiniPascalLexer.exe <arquivo_entrada.txt> <arquivo_saida.txt>

3. Respostas Fundamentadas às Decisões de Projeto (Página 2)

a) Limite para tamanho de identificadores?

Não há limitação artificial curta. O analisador suporta identificadores de tamanho padrão na indústria (até 255 caracteres) com armazenamento dinâmico de strings em memória sem degradação de performance.

b) Formato das cadeias de tokens:

Foi adotado o padrão exato demonstrado no exemplo oficial do PDF (páginas 3 e 4): cada linha de saída contém o par <lexema token_amigável> (exemplo: program Palavra reservada, x Identificador, := Atribuição, 3.1416 Número real).

c) Estrutura da Tabela de Palavras Reservadas:

A tabela foi implementada através da classe SymbolTable utilizando a estrutura Dictionary<string, TokenType> (Tabela Hash interna do C#). Isso garante complexidade de busca em tempo constante O(1). A tabela é inicializada na carga com as 47 palavras reservadas da linguagem com sensibilidade insensível a maiúsculas/minúsculas (OrdinalIgnoreCase).

d) Fase de inserção na Tabela de Símbolos:

O analisador léxico realiza o primeiro registro do identificador (lexema e primeira ocorrência). Os atributos contextuais complexos (tipo estático, escopo léxico, endereçamento de variáveis e assinatura de procedimentos) serão associados pelo analisador sintático/semântico na Unidade II.

e) Estratégia de tratamento de erros léxicos:

Foi adotada a recuperação por modo pânico (descarte local). O analisador não aborta a execução ao encontrar um caractere inválido (@, $, etc.) ou um literal/comentário malformado; ele registra o erro com linha e coluna no painel de diagnósticos, descarta o símbolo irregular e retorna ao estado q0 para continuar a análise.

4. Bateria de Testes e Validação Experimental

Arquivo de Teste	Objetivo / Cenário Testado	Resultado Obtido
`teste1_exemplo1.txt`	Exemplo 1 do PDF oficial: declarações var, const pi := 3.1416, if/else, read, write.	**50 tokens** reconhecidos. **0 erros.** Saída idêntica ao gabarito.
`teste2_exemplo2.txt`	Exemplo 2 do PDF oficial: programa 'Piloto', laço for cont=1 to total do, cálculo de médias e constantes strings.	**108 tokens** reconhecidos. **0 erros.**
`teste3_estruturas.txt`	Estruturas adicionais: laços while-do, repeat-until, constantes caractere ('A') e operadores lógicos.	**72 tokens** reconhecidos. **0 erros.**
`teste4_operadores.txt`	Operadores relacionais (<=, >=, <>), aritméticos (+, -, *, /, mod) e notação científica (24.40e-04).	**69 tokens** reconhecidos. **0 erros.**
`teste5_erros.txt`	Resiliência e recuperação de erros: caracteres inválidos (@, $), string não fechada, char longo e expoente sem dígitos.	**36 tokens** processados. **5 erros detectados** com linha e coluna.

5. Organização do Código-Fonte

O projeto está organizado modularmente no diretório MiniPascalLexer/:

- **Lexer/TokenType.cs:** Enumeração com as categorias de tokens e extensões para formatação amigável.
- **Lexer/Token.cs:** Entidade que encapsula Lexema, Tipo, Linha, Coluna e formatador de saída.
- **Lexer/SymbolTable.cs:** Tabela Hash com busca *O(1)* para as 47 palavras reservadas e operadores nomeados.
- **Lexer/AfdLexer.cs:** Motor de execução baseado nos estados e transições do AFD projetado.
- **UI/MainForm.cs:** Janela principal Windows Forms com painéis integrados e visualização em tempo real.
- **Program.cs:** Ponto de entrada híbrido (inicia a GUI sem parâmetros ou modo batch via CLI).